ENGINEERING
McMaster-Mohawk
Bachelor of Technology
Partnership

McMaster University

Artificial Intelligence and Machine Learning

Assignment 1: Logistic Regression

Note: Review the following sections for study purposes. Submit only the bug-free code for Section 4 to A2L in either .ipynb or .txt format by the due date.

Section 1 – Explore the Titanic dataset

Logistic regression is a commonly used algorithm that is capable of estimating the probability that a data point belongs to a specific class. For example, you want to predict whether a person has caught the novel coronavirus, and you set the classification threshold to 50%. If the estimated probability reached beyond your classification threshold, then the model predicts that the person tests positive. In other words, the instance belongs to the positive class, labeled “1”. And if the estimated probability is smaller than 50%, then it belongs to the negative class, labeled “0”. In general, logistic regression is a binary classifier.

But how does this algorithm work? Just like the model prediction of linear regression that we learned, firstly, we sum the product of each feature value by its corresponding coefficient and then add the bias term. The summed up result is called Log-odds. The equation of the log-odds(noted as z) is shown below:

$$z = \theta_0 + x_1\theta_1 + \cdots + x_n\theta_n$$

As mentioned before, the vectorized form of log-odds looks exactly like how we obtained prediction using linear regression:

$$z = x^T\theta$$

It might not be a good idea to throw out a lot of equations at this stage. Thus, let us take a look at the first dataset that we will be using today: the Titanic dataset. As you might have heard, the RMS Titanic set sail in April 1912, crossing the Atlantic Ocean from Southampton to New York. The ship is said to hit an enormous iceberg and sank into the ocean about 400 miles south of Newfoundland, Canada. There were 1502 out of 2224 passengers on board who lost their lives during this tragic event.

Kaggle provides the dataset that we will be using in this assignment. The training set contains some features of 891 passengers on board. We would write a Logistic Regression model and then use it to predict who survived the sinking based on features like age, ticket class, and so on.

1. Open the dataset named 'train.csv' which you downloaded from Avenue To Learn. If the dataset could not be downloaded, you can download it from Kaggle.

Take a look at this dataset. The following headers should be present:

- *survival*: whether the passenger survived the sinking, 0=No, 1=Yes
- *pclass*: Ticket class. 1=first class, 2=second class, 3=third class
- *sex*
- *Age*: age in years
- *ticket*: ticket number
- *parch*: number of parents/children aboard the Titanic

2. Create a new Python script either as a notebook(.ipynb) or using other IDE in a location that you prefer. Move the downloaded datasets into the same folder as your Python script.
   Import the following packages:

```
import pandas as pd
import numpy as np
```

   Load the train.csv dataset into pandas Dataframe object. We name it df. Use .head() method to get a sense of how the dataframe is structured.

```
df = pd.read_csv('./titanic/train.csv')
df.head()
```

3. Let us only explore the first 20 rows of this dataset for now.

```
df_20 = df.iloc[:20].copy()
```

   Use .info() to see if there are any missing values present in the features.

```
df.info()
```

   You might find out that there are missing values in the Age feature ('Cabin' will not be used as a feature, so ingnore it). Therefore, we can fill the blank rows with the mean value of the available ages.

```
mean = df_20['Age'].mean()
df_20['Age'].fillna(mean, inplace=True)
```

Section 2 – Walkthrough of the algorithm

4. Extract the Age feature into a column vector(2-D vector).

```
age = df_20[['Age']]
```

Extract the Survived feature into a 1-D vector.

```
survival = df_20['Survived']
```

5. As mentioned above, the equation for calculating log-odds is as follows:

$$z = \theta_0 + x_1\theta_1 + \cdots + x_n\theta_n$$

$$z = x^T\theta \quad \text{(vectorized form)}$$

In this equation, z stands for log-odds, x is your feature values, and θ is the model parameters(coefficients and the bias). Note that the bias is also called the intercept. For now, the coefficient and the bias term are given below. We would go through how to obtain these values later using Sci-kit Learn.

```
coef = np.array([[0.02705495]])
bias = np.array([-0.75660013])
```

As we can see, the log-odds is calculated as the dot product of x and θ. Let us write a function called log_odds. The parameters passed into this function would be features, coefficients, and bias. The function would return the dot product of features and coefficients plus the bias term.

```
def log_odds(features, coefficients, bias):
    return np.dot(features, coefficients) + bias
```

6. We can now calculate log-odds of surviving the sinking for those 20 passengers that we sampled out. Pass the coef, bias, and age variables that we defined in the last two steps into the function we just created. Store the returned output to a variable named result_log_odds.

```
result_log_odds = log_odds(age, coef, bias)
result_log_odds
```

Next, the log-odds would go through a sigmoid function (σ) to get the probability $\hat{p}$.

$$\hat{p} = h(z) = \sigma(x^T\theta)$$

The sigmoid function has an S-shape, and it maps the log-odds to a number between 0 and 1. It is defined this way:

$$h(z) = \frac{1}{1 + e^{-z}}$$

7. Let us define a sigmoid function called sigmoid() that takes log_odds as an argument. Inside the function, define a variable called the denominator. The denominator equals to 1 plus the exponential of negative log-odds.

```
def sigmoid(log_odds):
    denominator = 1 + np.exp(-log_odds)
    return 1 / denominator
```

8. Now let us calculate the estimated probabilities using this sigmoid function and the result_log_odds we obtained earlier. Store the result into a variable called p_hat.

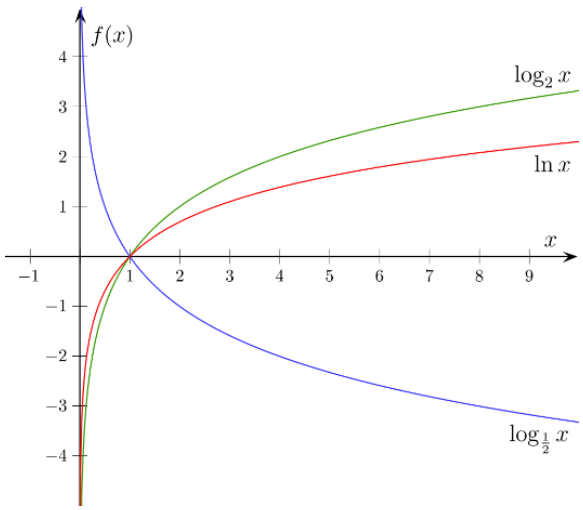
```
p_hat = sigmoid(result_log_odds)
```

The probability $\hat{p}$ estimated by the logistic model indicates how likely an instance belongs to the positive class.
In our case, the model predicts whether the passenger would survive the sinking of the Titanic. Here is our the model makes the prediction(noted as $\hat{y}$),
when the classification threshold is 0.5 :

$$\hat{y} = \begin{cases} 0, & \hat{p} < 0.5 \\ 1, & \hat{p} \geq 0.5 \end{cases}$$

When the estimated probability is greater than 0.5, the instance(data point) will be classified into the positive class(y=1). But how do we train the logistic regression model? In other words, what coefficients and bias do we use so the model would predict the survival of the passenger most accurately? For those positive instances(passenger did survive), the model ought to estimate high probabilities. And conversely, the model estimates low probabilities for negative instances.

The optimized values of the coefficient for the Age feature and the bias term were provided in Step 5. However, to get these values, a cost function called the log-loss would be needed. We have seen some cost functions for linear regression. The cost function is used to evaluate the performance of a machine learning model. The log-loss cost function over the whole training set looks quite intimidating. Thus, let us look at the cost function for a single instance first.



Take a look at the graph of the logarithmic functions shown above.

$$cost\ function = \begin{cases} -\log(\hat{p}), & if\ positive\ instance \\ -\log(1-\hat{p}), & if\ negative\ instance \end{cases}$$

The natural logarithm ln(x) approaches negative infinity when x approaches 0. Thus, similarly the -log($\hat{p}$) in the cost function would be very large if a positive instance has a probability close to 0.

Based on the graph, we can see that when x is close to 1, log(x) would be close to 0, so does -log(x). Therefore, when the model estimates a probability close to 0 for a negative instance, we get -log(1-0), the cost would be close to 0. Our objective is to compute the value of the parameter vector θ that minimizes the cost, so this logarithmic cost function suits our needs well.

Log-loss is the function that computes the average cost over the whole training instances, similar to the one shown earlier. The equation of the log-loss is shown here:

$$\begin{aligned} logloss = & -\frac{1}{m}\sum_{i=1}^{m}[y\log(\hat{p}) + (1-y)\log(1-\hat{p})] \\ = & -\frac{1}{m}\sum_{i=1}^{m}[y\log(h(z)) + (1-y)\log(1-h(z))] \end{aligned}$$

In this equation,

- m is the total number of instances
- y is the class of the ith instance, in our case, 0 or 1, according to the Survived feature values.
- z is the log-odds of the ith instance
- $h(z)$ is equivalent to the probability $\hat{p}$ that we calculated in Step 8.

9. Let us convert the log-loss equation into Python code. Create a function named log_loss that takes two arguments: probabilities and labels. The probabilities are derived from the p_hat variable that we obtained in Step 8. The labels are the ground truth values from survival variable, and it is used to compare with the predicted values.

```
def log_loss(probabilities, labels):
    return (-1 / labels.shape[0]) * np.sum(labels * np.log(probabilities) + (1-labels) *
np.log(1-probabilities))
```

We can calculate the cost with the function below, note that the log_loss function requires that the survival be a column vector here, hence the reshape:

```
survival = survival.to_numpy().reshape(-1, 1)
cost = log_loss(p_hat, survival)
```

10. Now we have our model ready to make predictions. In Step 5, the optimized values of the parameter vector θ were given beforehand. However, in practice, the Gradient Descent optimization algorithm would be used to find the best values that minimize the log-loss across the whole training set data.
Let us create a function called predict() that takes four arguments: features, coefficients, bias, threshold. The body of this function contains what we went through in the earlier steps. The log_odds function and the sigmoid function would be included. We will use np.where() method to return 1 if the probability exceeds 0.5 and vice versa.

```
def predict(features, coefficients, bias, threshold):
    result_log_odds = log_odds(features, coefficients, bias)
    p_hat = sigmoid(result_log_odds)
    return np.where(p_hat >= threshold, 1, 0)
```

Next, define a variable named predicted_result and invoke the predict function we just created.

```
predicted_result = predict(age, coef, bias, 0.5)
```

In the next section, we would rapidly create a logistic regression model using Sci-kit Learn, so we do not have to build the model from scratch.

Section 3 – Using Sci-kit Learn to create a Logistic Regression model

11. Import the sklearn package so we could instantiate a model instance from the LogisticRegression class

```
from sklearn.linear_model import LogisticRegression
```

Create a logistic regression model named logit_reg.

```
logit_reg = LogisticRegression()
```

12. Now before we fit the feature and label directly into the model, note that the data shall be normalized first. The reason for the normalization is because the logistic regression model from sklearn uses a technique called regularization internally. Sklearn requires normalized feature data, which spread within the same range with other features. Therefore, let us import a StandardScaler class from Sci-kit Learn's preprocessing module. StandardScaler would subtract the feature value by the mean value and then divide it by the standard deviation. It is also called the z-score normalization.

```
from sklearn.preprocessing import StandardScaler
```

Let us create an instance from the StandardScaler class and name it z_score_scaler. Then use fit_transform() method to scale the age values. Store the result as age_scaled.

```
z_score_scaler = StandardScaler()
age_scaled = z_score_scaler.fit_transform(age)
```

13. Train the logistic regression model using age_scaled and survival as the features and the labels, respectively.

```
logit_reg.fit(age_scaled, survival.ravel())
```

Now you can check the coefficient and the bias term from this model. If you pass in age_scaled, logit_reg.coef_, and logit_reg.intercept_ into the log_loss function we defined earlier, you will notice that the costs are the same.

```
print(logit_reg.coef_)
print(logit_reg.intercept_)
```

Now, you test if the parameter vector given in step 5 is the optimized one.

```
result_log_odds_sk = log_odds(age_scaled, logit_reg.coef_, logit_reg.intercept_)
p_hat_sk = sigmoid(result_log_odds_sk)
cost_sk = log_loss(p_hat_sk, survival)
print(cost_sk)
```

14. Remember, we only extracted 20 instances from the original training set that contains 891 instances. Thus, we can obtain the next 20 instances and feed it into the model to make predictions.

```
df_20_new = df.iloc[20:40].copy()
```

```
df_20_new.info()
```

Similarly, there are missing values in the Age feature. Therefore we can fill the blank spaces with the mean value. After that, we use z-score normalization to scale the data.

```
mean = df_20_new['Age'].mean()
df_20_new['Age'].fillna(mean, inplace=True)
age_new = df_20_new[['Age']]
age_new_scaled = z_score_scaler.fit_transform(age_new)
```

15. Make predictions with the logit_reg model we defined earlier. Compare it with the true label values.

    ```
    predictions = logit_reg.predict(age_new_scaled)
    print(predictions)
    print(df_20_new['Survived'].ravel())
    ```

As we can see, the predicted values are still quite different than the label values. We ought to try feeding more instances(data points) to train the logistic regression model. Apart from the Age feature, we could also include other features such as seat class, gender, and so on.

Section 4 – Submission
Using the whole dataset to train the Logistic Regression model; Submit the code for this section with either '.ipynb' format or '.txt' format to A2L.

1. Import the following libraries and then load the **train.csv** dataset into a Dataframe object.

   ```
   import pandas as pd
   import numpy as np
   from sklearn.linear_model import LogisticRegression
   from sklearn.preprocessing import StandardScaler
   from sklearn.model_selection import train_test_split
   ```

2. Transform the values under the Sex column to numeric values. You may use the .map() method.

   ```
   df['Sex'] = df['Sex'].map({'female':1, 'male':0})
   ```

3. Check if any values are missing by using the .info() method. The Age column likely has missing values, fill the blank spaces with the mean value.

4. Since logistic regression is a binary classifier, we could append three more columns into the original dataset, namely UpperClass, MiddleClass and LowerClass. Here is sample code for UpperClass:

   ```
   df['UpperClass'] = df['Pclass'].apply(lambda x: 1 if x == 1 else 0)
   ```

5. In the previous section, we only took the age feature into consideration while buiding the model. Now let us feed five features: Sex, Age, UpperClass, MiddleClass, LowerClass.

   ```
   X = df[['Sex', 'Age', 'UpperClass', 'MiddleClass', 'LowerClass']]
   ```

6. Define a variable y. Assign the value of the column ‘Survived’ as the label into this variable.

7. Let us do a train-test split to the data. Set the size of the test set to 20%.

   ```
   X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
   ```

8. Create a StandardScaler instance and name it scaler. After that, use the scaler to transform the feature values.

   ```
   X_train_scaled = scaler.fit_transform(X_train)
   X_test_scaled = scaler.transform(X_test)
   ```

9. Create a logistic regression model, fit the model with X_train_scaled and y_train.

10. Get the score on the test set. Print your score.

    ```
    score_test = logit_reg.score(X_test_scaled, y_test)
    ```

11. Here are some additional testing data from the movie Titanic directed by James Cameron in 1997. If you want, feel free to replace the last set of data with your data.

    ```
    Jack = np.array([0.0, 20.0, 0.0, 0.0, 1.0])
    Rose = np.array([1.0, 17.0, 1.0, 0.0, 0.0])
    Fabrizio = np.array([0.0, 22.0, 0.0, 0.0, 1.0])
    Student = np.array([0.0, 24.0, 0.0, 1.0, 0.0])
    ```

    Use the scaler we used in step 8 to do a z-score normalization.

    ```
    fictional_passengers = np.array([Jack, Rose, Fabrizio, Student])
    fictional_passengers_scaled = scaler.transform(fictional_passengers)
    ```

12. Predict the fictional_passengers_scaled data and print your result.

END